캡스톤디자인 결과보고서 (P2P)

NAT 환경을 위한 종단 간 암호화 P2P 통신 시스템 OpenP2P의 설계 및 구현

학번: 202311628 이름: 나르지기토브 예르도스 (Yerdos Narzhigitov)

Design and Implementation of OpenP2P: An End-to-End Encrypted Peer-to-Peer Communication System for NAT Environments

요 약

이 보고서는 NAT 뒤에 있는 두 사용자가 중앙 서버에 내용을 맡기지 않고 직접 통신하는 종단 간 암호화 P2P 시스템 OpenP2P의 설계와 구현을 다룬다. 랑데뷰 서버는 방 ID와 비밀번호로 두 피어를 소개만 하고, 피어는 TCP 동시 열기 기반 홀 펀칭으로 직접 연결한다. 연결 후에는 X25519 임시 키와 장기 신원 키를 BLAKE2b로 함께 섞는 인증 핸드셰이크를 수행하여, 전방 비밀성을 얻고 중개 서버의 공개키 바꿔치기를 탐지한다. 그 위에서 XChaCha20-Poly1305로 암호화된 채팅, 파일 전송, SOCKS5 프록시를 제공한다. 루프백 환경 시험에서 공개키 바꿔치기를 9회 모두 탐지하였고, 1 GiB 파일을 무결성 손상 없이 평균 658 MB/s로 전송하였다. 또한 홀 펀칭의 경쟁 조건을 찾아 수정함으로써 연결 성공률을 87%에서 100%로 높였다.

<키워드 : P2P, NAT 홀 펀칭, 종단 간 암호화, 전방 비밀성, 중간자 공격, libsodium>

학생: Narzhigitov Yerdos
지도교수: 인

I. 서 론

메신저와 파일 공유 서비스는 대부분 중앙 서버를 거쳐 데이터를 전달한다. 이 구조에서는 서버 운영자가 통신 내용이나 메타데이터에 접근할 수 있고, 서버가 장애나 검열의 단일 지점이 된다. 두 사용자가 직접 연결하는 P2P(Peer-to-Peer) 방식은 이런 문제를 줄이지만, 실제 인터넷에서는 두 가지 장벽에 부딪힌다.

첫째, 대부분의 가정과 모바일 단말은 NAT(Network Address Translation) 뒤에 있어 외부에서 먼저 연결을 맺을 수 없다. 둘째, 상대를 찾기 위해 중개 서버가 필요한데, 이 서버가 공개키를 바꿔치기하면 중간자 공격(MITM)이 가능하다. 즉, 단순히 직접 연결하는 것만으로는 기밀성이 보장되지 않는다.

본 프로젝트의 목표는 두 사용자가 방 ID와 비밀번호만 공유하면 NAT를 넘어 직접 연결하고, 중개 서버도 내용을 읽을 수 없는 종단 간 암호화(E2EE) 채팅과 파일 전송을 제공하는 시스템을 만드는 것이다. 이를 위해 C 언어와 libsodium을 사용하여 OpenP2P를 설계·구현하였다. OpenP2P는 랑데뷰(rendezvous) 서버와 피어(peer) 프로그램 두 개로 구성된다.

본 프로젝트의 주요 기여는 다음과 같다. 본인이 설계, 구현, 문서화, 테스트를 모두 단독으로 수행하였다(2026년 4월 27일~5월 26일, 커밋 123개, 약 6,000행).

- TCP 동시 열기(simultaneous open)를 이용한 홀 펀칭으로 NAT 뒤의 두 피어를 직접 연결하고, IPv4와 IPv6를 모두 지원한다.
- 임시 키와 장기 신원 키를 함께 묶는 인증 핸드셰이크로, 전방 비밀성을 유지하면서 랑데뷰 서버의 키 바꿔치기를 탐지한다.
- 파일마다 별도의 암호화 스트림을 쓰는 파일 전송, 전송 중 취소, 진행률 표시를 구현하였다.
- 평문 비밀번호 대입 공격을 막는 방별 시도 제한(5회)과 방 만료(5분)를 랑데뷰 서버에 넣었다.
- 호스트를 출구 노드로 쓰는 SOCKS5 프록시 모드를 추가하여, 암호화된 P2P 터널 위에서 일반 TCP 트래픽을 중계할 수 있게 하였다.

본 보고서의 구성은 다음과 같다. II장에서는 NAT 통과 기술과 기존 암호화 메신저를 살펴본다. III장에서는 시스템 구조, 프로토콜, 암호 설계, 구현 내용을 설명한다. IV장에서는 기능·보안·성능 시험 결과를 제시하고, V장에서 결론과 향후 과제를 정리한다.

II. 관련 연구

2.1 NAT 통과 기술

NAT는 사설 주소와 공인 주소를 변환하며, 내부에서 먼저 보낸 패킷에 대한 응답만 통과시킨다. 따라서 두 피어가 모두 NAT 뒤에 있으면 어느 쪽도 먼저 연결을 받을 수 없다. 이를 해결하는 대표적인 방법이 홀 펀칭(hole punching)이다 [1] [2].

홀 펀칭에서는 두 피어가 먼저 공인 서버에 접속하여 자신의 공인 주소와 포트를 알린다. 서버가 서로의 주소를 교환해 주면, 두 피어는 같은 로컬 포트에서 동시에 상대에게 연결을 시도한다. 양쪽 NAT에 매핑이 열려 있으므로 연결이 성립한다. Ford 등 [2]은 UDP뿐 아니라 TCP에서도 동시 열기를 이용한 홀 펀칭이 대부분의 NAT에서 동작함을 보였다. 다만 목적지마다 다른 외부 포트를 배정하는 대칭형(symmetric) NAT에서는 실패한다.

IETF는 이 과정을 표준화하였다. STUN [3]은 공인 주소를 알아내는 프로토콜이고, TURN [4]은 직접 연결이 실패할 때 중계 서버로 트래픽을 넘기는 프로토콜이다. ICE [5]는 여러 후보 경로를 수집하고 점검하여 가장 좋은 경로를 고른다. WebRTC와 Tailscale 같은 상용 시스템은 이 기법을 UDP 기반으로 조합한다. OpenP2P는 구현을 단순하게 유지하기 위해 랑데뷰 서버가 STUN 역할을 함께 맡고, TCP 동시 열기 방식만 사용하며, TURN 중계는 두지 않았다.

2.2 종단 간 암호화와 암호 기법

Signal 프로토콜 [6]은 X3DH 키 합의와 더블 래칫을 사용하여 전방 비밀성과 상대 인증을 함께 제공한다. X3DH의 핵심은 장기 신원 키와 임시 키의 Diffie-Hellman 결과를 섞어 세션 키를 만드는 것이다. 다만 Signal은 중앙 서버가 메시지를 중계하고 보관하는 구조이다. OpenP2P는 이 키 혼합 아이디어를 단순화하여 적용하되, 메시지는 피어 사이로만 오가고 어디에도 저장하지 않는다.

WireGuard [7]는 고정 공개키로 상대를 식별하고 Curve25519, ChaCha20-Poly1305, BLAKE2s 같은 현대적 암호 프리미티브만 사용하는 VPN이다. OpenP2P도 같은 철학에 따라 알고리즘 협상 없이 X25519 [8], XChaCha20-Poly1305, BLAKE2b [9]로 고정하였다. 이 프리미티브들은 libsodium [10]을 통해 사용한다.

Magic Wormhole [11]은 짧은 코드를 공유하여 두 컴퓨터 사이에 파일을 암호화 전송하는 도구로, 사용 방식이 OpenP2P의 '방 ID + 비밀번호'와 가장 비슷하다. Magic Wormhole은 PAKE로 짧은 코드를 강한 키로 바꾸고 중계 서버로 폴백한다. 반면 OpenP2P는 지속적인 신원 키와 핑거프린트로 상대를 확인하고, 채팅과 SOCKS5 터널까지 제공한다.

2.3 기존 접근과의 비교

시스템	연결 방식	서버가 내용을 볼 수 있는가	상대 확인	중계 폴백
일반 메신저	중앙 서버 중계	가능	계정	항상 중계
Signal [6]	중앙 서버 중계	불가 (E2EE)	안전 번호	항상 중계
Magic Wormhole [11]	직접 또는 중계	불가 (PAKE)	일회용 코드	있음
OpenP2P (본 프로젝트)	TCP 홀 펀칭 직접 연결	불가 (E2EE)	핑거프린트	없음

정리하면, OpenP2P는 중앙 저장 없이 직접 연결하고 중개 서버의 키 바꿔치기를 암호학적으로 탐지한다는 점에서 기존 도구와 구분된다. 대신 대칭형 NAT에서는 연결할 수 없다는 한계를 갖는다.

III. 주요 내용

3.1 요구사항

프로젝트 착수 시 다음 요구사항을 정의하였다.

구분	요구사항
기능	방 ID와 비밀번호만으로 두 피어가 서로를 찾는다
기능	NAT 뒤에 있는 두 피어가 직접 TCP 연결을 맺는다
기능	실시간 채팅, 임의 크기 파일 전송, 전송 취소를 지원한다
보안	모든 트래픽을 암호화하고 무결성을 검증한다
보안	랑데뷰 서버는 피어 간 내용을 복호화할 수 없다
보안	서버가 공개키를 바꿔치기하면 연결이 거부된다
보안	장기 키가 유출되어도 과거 세션은 복호화되지 않는다(전방 비밀성)
비기능	리눅스 표준 환경(gcc, libsodium, pthreads)만으로 빌드된다
비기능	채팅 기록을 디스크에 저장하지 않는다

3.2 시스템 구조

OpenP2P는 랑데뷰 서버 `rendezvous` 와 사용자 프로그램 `peer` 두 개의 실행 파일로 구성된다. 랑데뷰 서버는 두 피어를 소개한 뒤 연결에서 빠지며, 이후 모든 트래픽은 피어 사이를 직접 오간다. 하나의 세션은 아래 네 단계를 차례로 거친다.

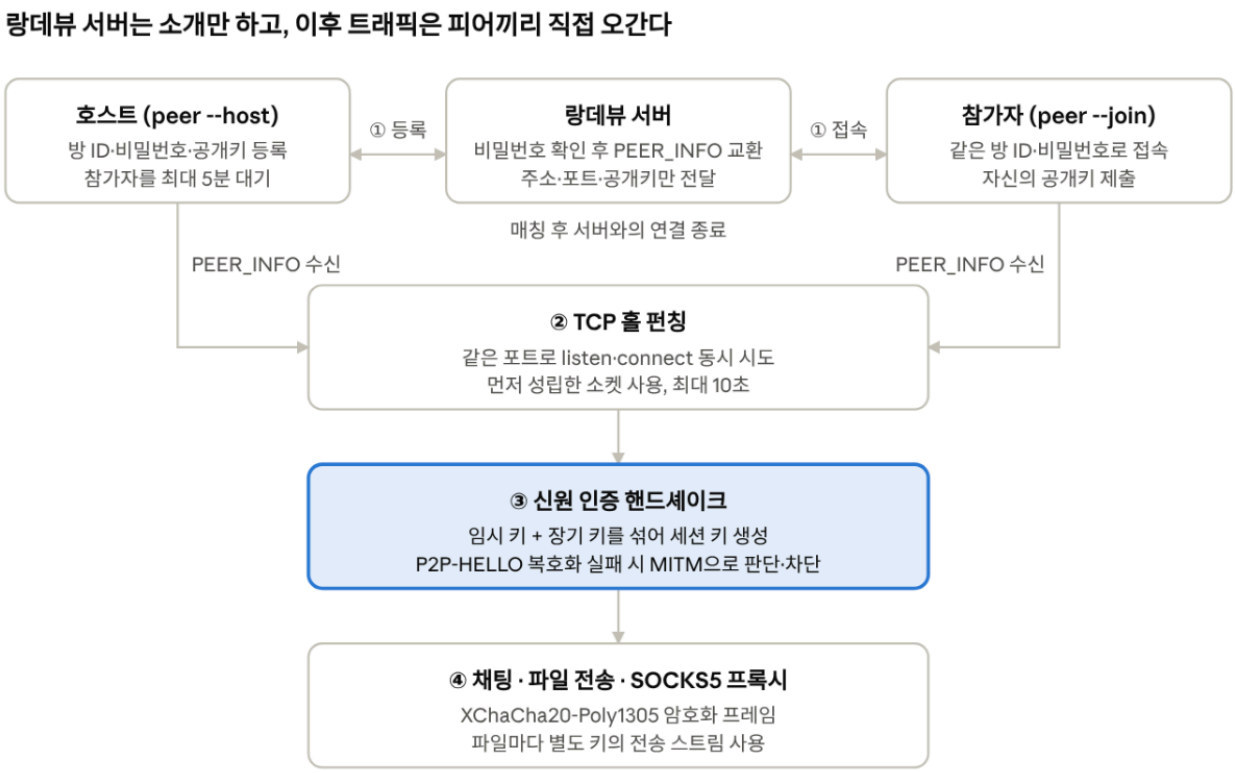


그림 1. OpenP2P 세션 수립 과정 · 4단계

랑데뷰 서버는 ① 단계에만 참여하며, ②~④ 단계는 두 피어 사이에서만 진행된다.

코드는 계층별로 나뉘며, 각 계층은 아래 계층만 알고 위 계층을 모른다. 예를 들어 암호 계층은 메시지 종류를 알지 못하고, 네트워크 계층은 암호화를 알지 못한다.

계층	모듈	역할
응용	`chat.c` , `file_stream.c` , `file_offer.c` , `progress.c` , `proxy.c` , `stream_table.c`	채팅, 파일 전송, 진행률 표시, SOCKS5 프록시
연결	`peer.c` , `holepunch.c` , `identity.c`	랑데뷰 절차, 홀 펀칭, 신원 키 관리
서버	`rendezvous.c` , `protocol.c` , `room.c`	접속 수락, 방 등록·매칭, 만료 처리
암호	`crypto.c`	X25519 키 교환, 암호화 프레임 송수신
네트워크	`net.c`	전체 송수신 보장, 소켓 바인딩
공통	`logger.c`	레벨별 로그

3.3 랑데뷰 프로토콜과 방 관리

피어는 랑데뷰 서버에 접속하자마자 익명 X25519 키 교환으로 암호화 채널을 만든다. 따라서 방 비밀번호는 네트워크에 평문으로 노출되지 않는다. 이후 서버가 역할을 물으면 피어는 역할(호스트 H / 참가자 J), 방 ID, 비밀번호, 장기 공개키를 순서대로 보낸다.

호스트가 등록하면 서버는 방 테이블에 호스트의 주소, 포트, 공개키, 암호 세션을 저장하고 대기한다. 참가자가 올바른 비밀번호로 방을 요청하면, 서버는 양쪽에 `PROTO_PEER_INFO` (주소 체계, IP, 포트, 32바이트 공개키)를 보내고 방을 비운다.

방 테이블은 뮤텍스로 보호되며, 등록·매칭·만료 정리가 같은 방을 동시에 건드리지 못하게 한다. 보안을 위해 다음과 같은 장치를 두었다.

- 비밀번호는 `sodium_memcmp` 로 비교하여 타이밍 공격을 막는다.
- 틀린 비밀번호가 들어오면 호스트에게 시도 횟수와 출발지 주소를 `PROTO_WARN` 으로 알리고, 5회 실패하면 방을 폐기한다.
- ‘방 없음’과 ‘비밀번호 오류’에 같은 오류 문구를 돌려주어 방 ID의 존재 여부를 노출하지 않는다.
- 참가자가 없는 방은 등록 5분 뒤 만료된다. 별도 스레드 없이 새 접속을 받을 때마다 정리한다.
- 방 수는 기본 5,000개로 제한한다.

서버는 접속마다 분리(detached) 스레드를 만들고, `IPV6_V6ONLY=0` 인 IPv6 소켓 하나로 IPv4와 IPv6 클라이언트를 모두 받는다. IPv4 주소는 `::ffff:` 매핑을 풀어 저장한다.

3.4 TCP 홀 펀칭

매칭 직후 피어는 랑데뷰 연결에 사용한 로컬 포트를 `getsockname()` 으로 알아낸 뒤 연결을 닫는다. 그리고 `SO_REUSEPORT` 로 같은 포트에 소켓 두 개를 만든다. 하나는 `listen()` 으로 상대의 연결을 기다리고, 다른 하나는 논블로킹 `connect()` 로 상대의 공인 주소에 연결을 시도한다.

두 소켓은 `poll()` 로 최대 10초간 감시하며, 먼저 성립한 쪽을 쓰고 나머지는 닫는다. 같은 포트를 재사용하므로 랑데뷰 접속 때 만들어진 NAT 매핑을 그대로 쓸 수 있다. 두 피어가 서로 다른 주소 체계(IPv4/IPv6)로 접속한 경우에는 매핑을 재사용할 수 없으므로 경고를 출력하고 일반 연결만 시도한다.

3.5 신원 인증 핸드셰이크

익명 키 교환만 쓰면 랑데뷰 서버가 `PROTO_PEER_INFO` 안의 공개키를 자신의 키로 바꿔 MITM을 할 수 있다. 이를 막기 위해 P2P 핸드셰이크는 임시 키와 장기 신원 키를 모두 세션 키에 반영한다.

1. 양쪽이 새 임시 X25519 키쌍을 만들고 공개키를 교환한다.
2. 장기 공개키를 바이트 단위로 비교하여 클라이언트/서버 역할을 정한다. 양쪽이 같은 결과를 얻으므로 별도 협상이 필요 없다.
3. 임시 키끼리 키 교환하여 k1(전방 비밀성), 장기 키끼리 키 교환하여 k2(인증)를 얻는다.
4. 네 개의 절반 키를 정렬한 뒤 BLAKE2b로 해시하여 하나의 세션 키를 만든다.
5. 양쪽이 `P2P-HELLO` 를 암호화하여 보내고, 상대의 메시지가 복호화되는지 확인한다.

$$K_{session} = \mathrm{BLAKE2b}\big(\mathrm{sort}(k_1^{rx},\ k_1^{tx},\ k_2^{rx},\ k_2^{tx})\big)$$

공개키가 바뀐 경우 공격자는 장기 비밀키가 없으므로 올바른 k2를 계산할 수 없다. 그 결과 양쪽의 세션 키가 달라지고 첫 `P2P-HELLO` 복호화가 실패하여 연결이 거부된다. 또한 임시 비밀키는 핸드셰이크 직후 `sodium_memzero` 로 지우므로, 나중에 장기 키가 유출되어도 과거 세션의 k1을 복원할 수 없다.

장기 키는 `~/.config/openp2p/identity.key` 에 64바이트(공개키 32 + 비밀키 32)로 저장한다. 파일은 `O_EXCL` 과 0600 권한으로 만들고, 그룹이나 타인이 읽을 수 있으면 SSH처럼 로드를 거부한다. 파일이 손상되면 자동으로 새 키를 만들지 않고 실행을 멈춰, 신원이 조용히 바뀌는 것을 막는다. 사용자는 화면에 표시되는 핑거프린트(예: `8f3a 92b1 c4d7 e8f0`)를 전화 등 별도 채널로 대조하여 상대를 확인한다.

3.6 암호화 메시지 형식

모든 메시지는 libsodium의 secretstream(XChaCha20-Poly1305)으로 암호화된다. 하나의 프레임은 4바이트 길이 접두사와 암호문으로 이루어지고, 평문의 첫 바이트가 메시지 종류이다. 종류 바이트가 암호문 안에 있으므로, 공격자가 이를 바꾸면 인증 태그 검증에서 실패한다. secretstream은 내부 카운터를 사용하므로 프레임의 재전송이나 순서 변경도 탐지된다.

필드	크기	설명
length	4바이트	암호문 길이(네트워크 바이트 순서)
ciphertext	평문 + 17바이트	종류(1바이트) + 페이로드를 암호화한 값

프레임 크기는 최대 1 MiB로 제한하며, 이보다 큰 길이를 선언하는 상대는 메모리 할당 전에 연결을 끊는다. 메시지 종류는 범위별로 나뉘어 있다: 0x01~0x0F 채팅·제어, 0x10~0x1F 랑데뷰, 0x20~0x2F 파일 전송, 0x30~0x3F 프록시.

3.7 채팅과 파일 전송

피어는 두 개의 스레드로 동작한다. 메인 스레드는 터미널을 raw 모드로 두고 키 입력을 한 글자씩 읽으며, 수신 스레드는 상대의 메시지를 받아 종류별로 처리한다. 입력 중인 줄이 들어온 메시지에 가려지지 않도록 화면을 다시 그리며, 입력 상태용과 출력용 뮤텍스 두 개를 항상 같은 순서로 잠가 교착 상태를 피한다.

파일 전송은 `/send <경로>` 명령으로 시작한다.

1. 송신자가 파일 이름과 크기를 `MSG_FILE_OFFER` 로 보낸다.
2. 수신자는 `y/n` 으로 수락 여부를 답한다.
3. 수락하면 송신자는 이 파일만을 위한 새 키와 헤더를 만들어 제어 채널로 보낸다.
4. 파일을 64 KiB 조각으로 나눠 전송 스트림으로 보내고, 마지막 조각에 `TAG_FINAL` 을 붙인다.
5. 수신자는 완료 후 `MSG_TRANSFER_DONE` 을 보내고 채팅 모드로 돌아간다.

`TAG_FINAL` 이 있어야만 전송이 완료된 것으로 보므로, 중간에 잘린 파일을 완성본으로 오인하지 않는다. 수신 측은 파일 이름에서 경로를 제거하고, `.` 으로 시작하거나 제어 문자가 들어간 이름을 거부하여 경로 조작 공격을 막는다. 같은 이름의 파일이 있으면 `report_1.pdf` 처럼 번호를 붙여 덮어쓰지 않는다.

전송 중 Ctrl-C를 누르면 시그널 핸들러가 취소 플래그만 세운다. 송신 루프는 다음 조각에서 이를 보고 `TAG_FINAL` 로 스트림을 닫으므로, 프로세스를 종료하지 않고 채팅으로 돌아갈 수 있다. 진행 표시줄은 터미널 너비에 맞춰 전송량, 속도, 남은 시간을 보여준다.

3.8 SOCKS5 프록시 모드

양쪽이 `--proxy` 옵션을 주면 채팅 대신 프록시 모드로 동작한다. 연결 직후 `MSG_MODE` 로 양쪽 모드를 비교하며, 다르면 연결을 종료한다. 참가자는 `127.0.0.1:1080` 에 SOCKS5 서버를 열고, 호스트는 출구 노드가 되어 실제 목적지에 접속한다.

하나의 암호화 터널 위에 여러 TCP 연결을 실기 위해 각 연결에 32비트 스트림 ID를 붙이고, `OPEN` , `OPEN_OK/FAIL` , `DATA` , `CLOSE` 메시지로 다중화한다. 스트림 테이블은 최대 512개 연결을 관리하며, 한쪽 방향만 닫는 half-close를 지원한다. 여러 스레드가 터널에 동시에 쓰면 secretstream의 논스 카운터가 깨지므로, 모든 송신은 하나의 뮤텍스로 직렬화하였다. SOCKS5 주소는 IPv4, IPv6, 도메인 이름을 모두 받는다 [12].

3.9 개발 과정

개발은 Git으로 관리하였으며, 기능마다 `feat/crypto` , `feat/rendezvous-auth` , `feat/proxy` 같은 브랜치를 만들고 병합하는 방식으로 진행하였다. 빌드는 Makefile로 하며, `make asan` 은 AddressSanitizer와 스택 보호를 켜서 메모리 오류를 찾는 데 사용하였다. 실제로 핑거프린트 버퍼 크기 오류(19바이트)로 인한 오버플로를 이 과정에서 발견하여 수정하였다.

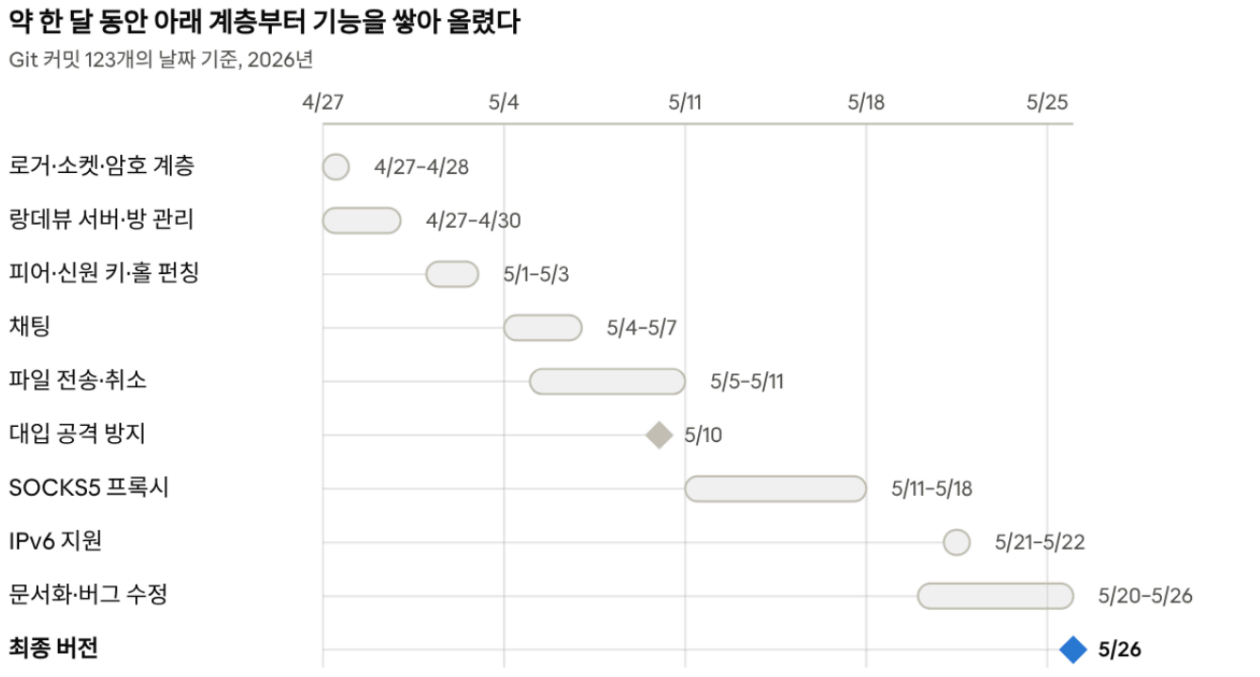


그림 2. 개발 일정 · 2026년 4월 27일~5월 26일

네트워크·암호 계층과 랑데뷰 서버를 먼저 완성한 뒤, 그 위에 피어 연결과 응용 기능을 차례로 올렸다.

IV. 결과 평가

기능·보안 시험은 모두 통과하였고, 암호화 파일 전송은 1 GiB 기준 평균 658 MB/s를 기록하였다. 다만 홀 펀칭에서 연결 실패율 13%의 경쟁 조건을 발견하였고, 이를 수정하여 100회 중 100회 성공으로 개선하였다.

4.1 시험 환경

항목	내용
운영체제	Ubuntu 24.04 (Linux 커널 6.18), 2 vCPU
컴파일러 / 라이브러리	gcc 13.3, libsodium 1.0.18, `-O2` 빌드
네트워크	한 대의 컴퓨터에서 루프백(127.0.0.1)으로 랑데뷰 서버 1개와 피어 2개 실행
자동화	Python `pexpect` 로 가상 터미널을 만들어 입력을 주고 출력을 검사
무결성 확인	원본과 수신 파일의 SHA-256 해시 비교

시험 환경의 커널에는 IPv6가 없어 랑데뷰 서버의 IPv6 소켓 생성이 실패하였다. 시험용 복사본에만 IPv4 대체 코드를 넣어 진행하였으며, 이 점은 4.6절의 한계에 포함하였다. 루프백 환경에는 NAT가 없으므로, 아래 결과는 프로토콜과 구현의 정확성·성능을 보여주며 실제 NAT 통과율을 나타내지는 않는다.

4.2 기능 시험

시험 항목	방법	결과
연결 수립	방 생성 후 참가자 접속, 100회 반복	87회 성공, 수립 시간 중앙값 103 ms (4.5절 참조)
채팅	메시지 100개 전송	전부 수신, 표시까지 지연 중앙값 0.29 ms (p95 0.54 ms)
파일 전송	1 MB, 10 MB, 100 MB, 1 GiB 전송	모두 SHA-256 일치
이름 충돌	같은 파일을 두 번 전송	`file_1MB_1.bin` 으로 저장, 기존 파일 유지
수신 거부	수신자가 `n` 입력	양쪽에 거부 표시, 채팅 계속
전송 취소	1 GiB 전송 중 송신자 Ctrl-C	‘Transfer cancelled’ 표시 후 채팅 정상 동작
모드 불일치	한쪽만 `--proxy` 사용	‘Mode mismatch’ 오류 후 종료
SOCKS5 프록시	curl로 100 MB 다운로드 3회	모두 SHA-256 일치
프록시 동시 연결	10 MB 다운로드 32개 동시 실행	32개 모두 성공, 1.24초
프록시 오류	열리지 않은 포트로 접속	curl이 연결 실패(코드 7)를 정상 보고
한글 채팅	‘안녕하세요’ 입력	**실패**: 비ASCII 문자가 입력 단계에서 버려짐 (4.6절)

4.3 보안 시험

중간자 공격 탐지. 랑데뷰 서버가 `PROTO_PEER_INFO` 의 공개키를 임의의 키로 바꿔 보내도록 수정한 시험용 서버를 만들었다. 10회 시도 중 홀 펀칭에 성공한 9회 모두 양쪽에서 `secretstream_pull() failed: forged or corrupted` 와 'AUTHENTICATION FAILED'가 출력되었고, 채널은 한 번도 수립되지 않았다. 나머지 1회는 4.5절의 홀 펀칭 실패였다. 3.5절에서 설명한 키 혼합 방식이 의도대로 동작함을 확인하였다.

비밀번호 대입 공격. 틀린 비밀번호로 5회 접속하자 참가자는 매번 'Authentication failed'를 받았다. 호스트에는 'Attempt 1: from 127.0.0.1:42519'부터 'Attempt 5'까지 경고가 표시된 뒤 'Closing…'으로 방이 폐기되었다. 이후 올바른 비밀번호로 접속해도 'No such room'이 반환되어, 방당 대입 기회가 5회로 제한됨을 확인하였다.

메모리 안전성. AddressSanitizer 빌드(`make asan`)로 채팅, 100 MB·10 MB 양방향 전송, 정상 종료까지 수행하였고, 피어와 랑데뷰 서버 모두에서 메모리 오류가 보고되지 않았다.

4.4 성능 시험

각 항목은 3회 측정한 평균이며, 비교를 위해 암호화 없는 기준값을 함께 측정하였다.

측정 항목	처리량 (MB/s)	비고
평문 TCP 전송 1 GiB (기준값)	2,438	암호화 없는 루프백 전송
OpenP2P 파일 전송 1 GiB	658	기준값의 27%
OpenP2P 파일 전송 100 MB	498	수락 후 시작 비용이 상대적으로 큼
HTTP 직접 다운로드 100 MB (기준값)	1,024	프록시 없음
SOCKS5 프록시 경유 100 MB	510	기준값의 50%
SOCKS5 프록시 32개 동시 (합계)	257	10 MB × 32

루프백에서는 네트워크 비용이 거의 없으므로, 기준값과의 차이는 암호화와 프레임 처리에 드는 CPU 비용이다. 1 Gbps 회선의 최대 속도는 약 125 MB/s이므로, 실제 인터넷 환경에서는 암호화가 병목이 되지 않을 것으로 판단된다. 프록시의 동시 연결 처리량이 낮은 것은 모든 스트림이 하나의 송신 뮤텍스와 하나의 TCP 연결을 공유하기 때문이다.

4.5 연결 신뢰성 문제와 개선

반복 시험 중 연결이 간헐적으로 실패하여 디버그 로그(`-L debug`)를 분석하였다. 실패한 쪽에서는 항상 `connect() immediately failed: Cannot assign requested address` (EADDRNOTAVAIL)가 출력되었다.

원인은 다음과 같다. 상대의 SYN이 먼저 도착하면 커널에 이미 같은 4-튜플(로컬 포트, 상대 주소·포트)의 연결이 생긴다. 이때 같은 튜플로 `connect()` 를 호출하면 즉시 실패하는데, 기존 코드는 이를 치명적 오류로 보고 `listen` 소켓까지 닫아 버렸다. 그런데 그 `listen` 소켓에는 바로 상대의 연결이 대기하고 있었다. 상대는 연결이 끊기거나(reset) 10초 뒤 시간 초과로 실패하였다.

수정은 `connect()` 가 즉시 실패해도 중단하지 않고 `listen` 소켓만 계속 감시하도록 한 것이다. `poll()` 은 음수 fd를 무시하므로 다른 코드는 바꿀 필요가 없다.

```
// holepunch.c - 수정 전
int connect_fd = open_connect(peer_family, local_port, peer_ip, peer_port);
if (connect_fd < 0) {
        close(listen_fd);
        return -1;
}

// 수정 후: 즉시 실패(EADDRNOTAVAIL 등)해도 listen 소켓으로 계속 대기
int connect_fd = open_connect(peer_family, local_port, peer_ip, peer_port);
```

빌드	연결 성공 (100회)	수립 시간 중앙값
수정 전	87회	103 ms
수정 후	100회	103 ms

이 문제는 두 피어가 서로 직접 도달할 수 있는 경우(같은 LAN, 공인 IP, 같은 컴퓨터)에 가장 자주 나타나지만, 상대의 SYN이 먼저 도착하는 모든 경우에 발생할 수 있다.

4.6 한계

- **대칭형 NAT와 중계 부재**: TURN 같은 중계 경로가 없어, 대칭형 NAT나 IPv4–IPv6 혼합 환경에서는 연결할 수 없다.
- **실제 NAT 환경 미측정**: 본 시험은 루프백에서 수행하였으므로, 통신사·가정용 공유기 등 실제 NAT에서의 통과율은 측정하지 못하였다.
- **비ASCII 입력 불가**: 채팅 입력은 ASCII 32~126만 받아(`chat.c` 654행) 한글 등 UTF-8 문자가 조용히 버려진다.
- **IPv6 없는 서버**: 랑데뷰 서버는 IPv6 소켓만 사용하므로, IPv6가 꺼진 호스트에서는 시작되지 않는다.
- **짧은 핑거프린트**: 핑거프린트는 공개키 앞 8바이트(64비트)를 그대로 표시한다. 해시를 사용하고 길이를 늘리면 위조 난이도가 높아진다.
- **전송 중 채팅 불가**: 제어 스트림과 전송 스트림을 번갈아 쓰므로, 파일을 주고받는 동안에는 채팅 입력이 무시된다.
- **이어받기 없음**: 중단된 전송은 처음부터 다시 시작해야 한다.
- **자동 시험 부재**: 저장소에는 자동 시험 코드가 없으며, 본 보고서의 시험 스크립트는 별도로 작성하였다.

V. 결 론

본 프로젝트에서는 중앙 서버에 내용을 맡기지 않고 두 사용자가 직접 통신하는 종단 간 암호화 P2P 시스템 OpenP2P를 C 언어로 설계·구현하였다. 랑데뷰 서버는 방 ID와 비밀번호로 두 피어를 소개만 하고, 피어는 TCP 홀 펀칭으로 직접 연결한 뒤 채팅, 파일 전송, SOCKS5 프록시를 암호화된 채널 위에서 제공한다.

핵심 성과는 임시 키와 장기 신원 키를 함께 섞는 인증 핸드셰이크이다. 이 방식으로 전방 비밀성을 유지하면서, 중개 서버가 공개키를 바꿔치기하는 공격을 시험에서 9회 모두 탐지하였다. 파일 전송은 1 GiB까지 무결성이 유지되었고 루프백에서 평균 658 MB/s를 기록하여, 일반 인터넷 회선 속도를 충분히 넘어섰다. 또한 반복 시험으로 홀 펀칭의 경쟁 조건을 찾아 연결 성공률을 87%에서 100%로 끌어올린 수정을 도출하였다.

향후 과제는 다음과 같다.

1. 4.5절의 홀 펀칭 수정을 본 저장소에 반영하고, 초기 SYN이 버려질 때를 대비해 `connect()` 재시도를 추가한다.
2. 채팅 입력에서 UTF-8을 지원하여 한글 메시지를 보낼 수 있게 한다.
3. 서로 다른 통신사·공유기 환경에서 실제 NAT 통과율을 측정하고, 실패 시 중계 서버(TURN 방식)로 넘어가는 경로를 추가한다.
4. 랑데뷰 서버에 IPv4 전용 대체 소켓을 두고, 핑거프린트를 공개키의 해시로 바꾼다.
5. 스트림 ID 기반 다중화를 파일 전송에도 적용하여 전송 중에도 채팅이 가능하게 하고, 본 보고서의 시험 스크립트를 자동 시험으로 저장소에 포함한다.

이 프로젝트를 통해 소켓 프로그래밍, NAT의 동작, 암호 프로토콜 설계, 멀티스레드 동기화를 하나의 시스템 안에서 연결하여 익힐 수 있었다. 특히 암호화만으로는 보안이 완성되지 않고, 누구와 통신하는지 확인하는 인증이 함께 설계되어야 한다는 점을 확인하였다.

VI. 참 고 문 헌

1. P. Srisuresh, B. Ford, D. Kegel, “State of Peer-to-Peer (P2P) Communication across Network Address Translators (NATs),” RFC 5128, IETF, 2008. link
2. B. Ford, P. Srisuresh, D. Kegel, “Peer-to-Peer Communication Across Network Address Translators,” USENIX Annual Technical Conference, 2005. arXiv:cs/0603074
3. M. Petit-Huguenin et al., “Session Traversal Utilities for NAT (STUN),” RFC 8489, IETF, 2020. link
4. T. Reddy et al., “Traversal Using Relays around NAT (TURN),” RFC 8656, IETF, 2020. link
5. A. Keranen, C. Holmberg, J. Rosenberg, “Interactive Connectivity Establishment (ICE),” RFC 8445, IETF, 2018. link
6. M. Marlinspike, T. Perrin, “The X3DH Key Agreement Protocol,” Signal Specifications, Revision 1, 2016. link
7. J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” NDSS Symposium, 2017. link
8. A. Langley, M. Hamburg, S. Turner, “Elliptic Curves for Security,” RFC 7748, IETF, 2016. link
9. M-J. Saarinen, J-P. Aumasson, “The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC),” RFC 7693, IETF, 2015. link
10. F. Denis et al., “libsodium documentation.” link
11. B. Warner et al., “Magic Wormhole documentation.” link
12. M. Leech et al., “SOCKS Protocol Version 5,” RFC 1928, IETF, 1996. link

---

학생: Narzhigitov Yerdos
지도교수: 인